```
git clone https://github.com/pjanczyk/scala-days-2026
```

VIRTUSLAB

Build a production service with VirtusLab Scala Stack and AI

Direct-style Scala 3: Tapir, Ox, Parlance, sttp.
No Futures, no IO.

Piotr Janczyk



Direct style: the code reads the way it runs

Futures (Scala standard library)

```scala
import scala.concurrent.Future
import scala.concurrent.ExecutionContext

given ExecutionContext = ExecutionContext.global

def report(id: Int): Future[Report] =
  for
    user <- findUser(id)
    report <- generateReport(user)
  yield report
```

Direct style

```scala
// no dependencies



def report(id: Int): Report =
  generateReport(findUser(id))
```



Direct style: the code reads the way it runs

Futures (Scala standard library)

```scala
import scala.concurrent.Future
import scala.concurrent.ExecutionContext

given ExecutionContext = ExecutionContext.global

def report(id: Int): Future[Report] =
  for
    (user, config) <- findUser(id).zip(findConfig())
    report <- generateReport(user, config)
  yield report
```



```scala
// no dependencies



def report(id: Int): Report =
  generateReport(findUser(id))
```



Direct style: ~~the code reads the way it runs~~ X

Futures (Scala standard library)

```
import scala.concurrent.Future
import scala.concurrent.ExecutionContext

given ExecutionContext = ExecutionContext.global

def report(id: Int): Future[Report] =
  for
    (user, config) <- findUser(id).zip(findConfig())
    report <- generateReport(user, config)
  yield report
```

Structured concurrency (JDK 21+ standard library)

```
import java.lang.Thread
import java.util.concurrent.Executors

val threadFactory = Thread.ofVirtual().factory()

def report(id: Int): Report =
  val (user, config) =
    Using.resource(
      Executors.newThreadPerTaskExecutor(threadFactory)
    ): executor =>
      val userF = executor.submit(() => findUser(id))
      val configF = executor.submit(() => findConfig())
      (userF.get(), configF.get())
  generateReport(user, config)
```



Direct style: the code reads the way it runs?

Futures (Scala standard library)

```scala
import scala.concurrent.Future
import scala.concurrent.ExecutionContext

given ExecutionContext = ExecutionContext.global

def report(id: Int): Future[Report] =
  for
    (user, config) <- findUser(id).zip(findConfig())
    report <- generateReport(user, config)
  yield report
```

Structured concurrency (JDK 21+ standard library)

```scala
import java.lang.Thread
import java.util.concurrent.Executors

val threadFactory = Thread.ofVirtual().factory()

def report(id: Int): Report =
  val (user, config) = par(findUser(id), findConfig())
  generateReport(user, config)

def par[A, B](a: =>A, b: =>B): (A, B) =
  Using.resource(
    Executors.newThreadPerTaskExecutor(threadFactory)
  ): executor =>
    val aTask = executor.submit(() => a)
    val bTask = executor.submit(() => b)
    (aTask.get(), bTask.get())
```



Direct style: the code reads the way it runs?

Futures (Scala standard library)

```
import scala.concurrent.Future
import scala.concurrent.ExecutionContext

given ExecutionContext = ExecutionContext.global

def report(id: Int): Future[Report] =
  for
    (user, config) <- findUser(id).zip(findConfig())
    report <- generateReport(user, config)
  yield report
```

Structured concurrency (JDK 21+ and Ox)

```
import java.lang.Thread
import ox.{supervised,fork,setOxThreadFactory}

setOxThreadFactory(Thread.ofVirtual().factory())

def report(id: Int): Report =
  val (user, config) = par(findUser(id), findConfig())
  generateReport(user, config)

def par[A, B](a: =>A, b: =>B): (A, B) =

  supervised:

    val aTask = fork(a)
    val bTask = fork(b)
    (aTask.join(), bTask.join())
```



Direct style: the code reads the way it runs?

Futures (Scala standard library)

```
import scala.concurrent.Future
import scala.concurrent.ExecutionContext

given ExecutionContext = ExecutionContext.global

def report(id: Int): Future[Report] =
  for
    (user, config) <- findUser(id).zip(findConfig())
    report <- generateReport(user, config)
  yield report
```

Structured concurrency (JDK 21+ and Ox)

```
import java.lang.Thread
import ox.{par,setOxThreadFactory}

setOxThreadFactory(Thread.ofVirtual().factory())

def report(id: Int): Report =
  val (user, config) = par(findUser(id), findConfig())
  generateReport(user, config)
```



Direct style: the code reads the way it runs ✔

Futures (Scala standard library)

```scala
import scala.concurrent.Future
import scala.concurrent.ExecutionContext

given ExecutionContext = ExecutionContext.global

def report(id: Int): Future[Report] =
  for
    (user, config) <- findUser(id).zip(findConfig())
    report <- generateReport(user, config)
  yield report
```

Structured concurrency (JDK 21+ and Ox)

```scala
import ox.par



def report(id: Int): Report =
  val (user, config) = par(findUser(id), findConfig())
  generateReport(user, config)
```

Virtual threads make blocking cheap.
Ox makes them easy to use with **par** and others.



Direct style: the code reads the way it runs

Futures (Scala standard library)

```
def report(id: Int): Future[Either[NotFound, Report]] =
  lookupLink(id).flatMap: // Future[Option[Link]]
    case None =>
      Future.successful(Left(NotFound(id)))
    case Some(link) =>
      lookupCountries(link)
        .zip(lookupReferrers(link))
        .map((countries, referrers) =>
          Right(Report(countries, referrers))
        )
```

EitherT (cats-effect)

```
def report(id: Int): EitherT[IO, NotFound, Report] =
  lookupLink(id) // OptionT[IO, Link]
    .toRight(NotFound(id))
    .semiflatMap: link =>
      (lookupCountries(link), lookupReferrers(link))
        .parMapN(Report(_, _))
```

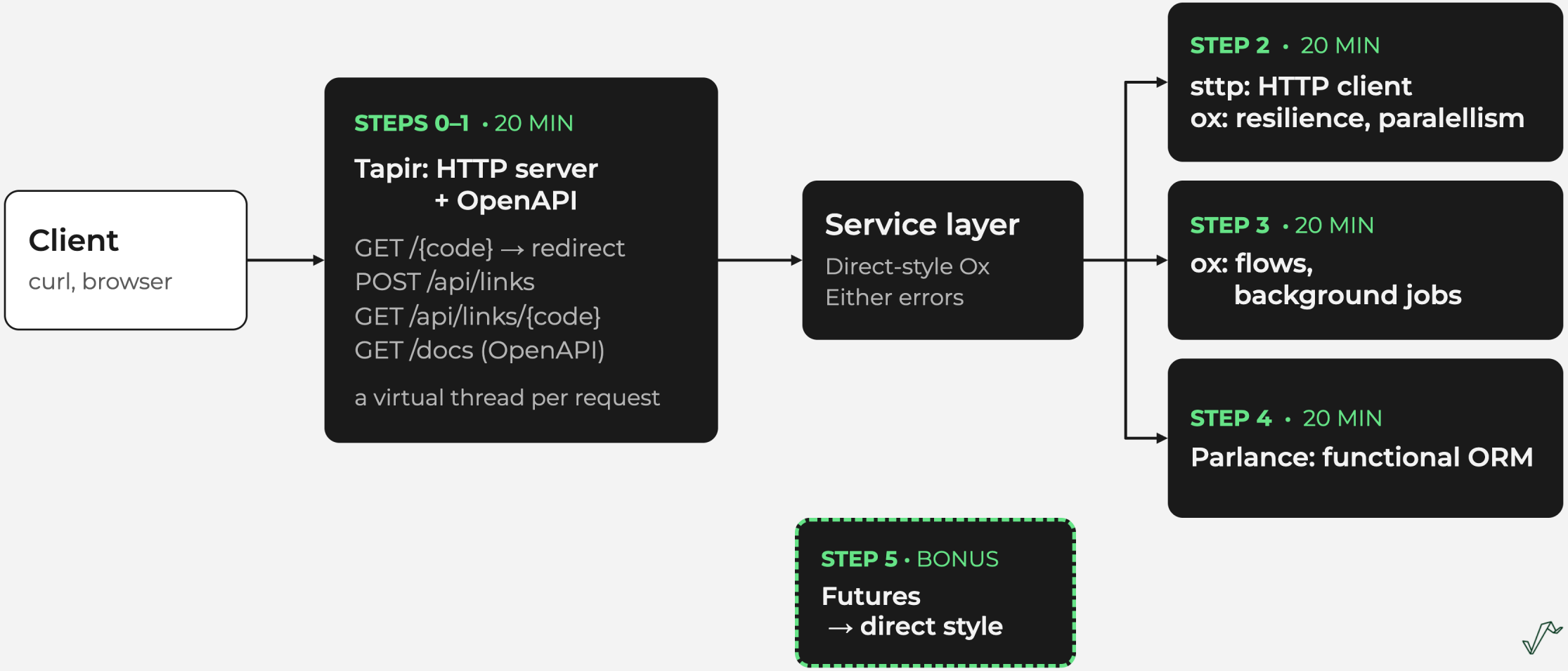
Structured concurrency (JDK 21+ and Ox)

```
def report(id: Int): Either[NotFound, Report] =
  lookupLink(id) // Option[Link]
    .toRight(NotFound(id))
    .map: link =>
      val (countries, referrers) = par(
        lookupCountries(link),
        lookupReferrers(link)
      )
      Report(countries, referrers)
```



/ WHAT WE BUILD

A link shortener with real-world problems



/ ALREADY WIRED IN FOR YOU

Production features already in place

OpenAPI

Swagger UI generated from the same Tapir endpoint definitions as the server

`GET /docs`

Metrics

Prometheus via a Tapir interceptor: requests, status codes and latency per endpoint

`GET /metrics`

Tracing

OpenTelemetry: a span per request, trace IDs in logs and carried across HTTP requests

`[INFO][trace 4bf92f35]`

Migrations

Flyway: versioned SQL files, applied once and in order on startup

`V1__create_link.sql`

Out of scope: Authentication, Healthchecks, DDD/Clean/Hexagonal architecture.

/ HOW THIS LAB WORKS · STEP 0 · 10 MINUTES

Your turn: run the app and pass the first test

1. Clone the repository

```
git clone https://github.com/pjanczyk/scala-days-2026
```

2. Run the app, then open **localhost:8080/docs**

```
scala-cli run .
```

3. Fill in the TODO in **HelloApi.scala**:
GET /api/hello?name=Berlin → "Hello, Berlin!"

```
endpoint.get.handleSuccess(name => ???)
```

4. Make the test pass

```
scala-cli test . --test-only '*Step0*'
```

More details in **steps/0-setup.md**

Every step

1. Implement the TODOs, following **steps/N-....md**
2. Make the step's test green
3. Commit your work, switch to the next step

```
git add -A
git commit -m "My step 0"
git switch step-1
```

Your work stays on your branch; everyone starts each step from the same code.

Done early? Stretch goal

Blocked? Ask any time

AI: Teach it the idioms with the **direct-style-scala** skill:
github.com/VirtusLab/scala-skill



Tapir: self-documenting APIs

```
val getLinkEndpoint =
  endpoint.get
    .description("Returns a link with title, visit count")
    .in("api" / "links" / path[String]("code"))
    .out(jsonBody[Link])
    .errorOut(oneOf[NotFound](notFound))
    .handle(code => service.getLink(code))
                                ^ Either[NotFound, Link]

NettySyncServer().port(8080)
  .addEndpoint(getLinkEndpoint)
  .startAndWait()
```

The types follow the **HTTP contract**: input, error, output.

Errors map to status codes with **oneOf**.

Built-in support for **OpenAPI**, **metrics**, **tracing**.

Supports direct-style (Netty), Futures (Pekko HTTP), ZIO, and more.



/ STEP 1 · 15 MINUTES

Your turn: finish the API in LinkApi.scala

steps/1-tapir.md

1. **createLink**: add a **conflict** variant (409) next to **.errorOut**
2. **getLink**: handled by service.getLink, with a **notFound** variant (404)
3. **redirect**: 302 and the URL in the Location header, handled by service.visitLink

Endpoint	Success	Errors
POST /api/links	201 + Link	400 invalid, 409 taken
GET /api/links/{code}	200 + Link	404
GET /{code}	302 + Location	404

Stretch: GET /api/links, listing all links

```
scala-cli test . --test-only '*Step1*'
scala-cli run .
curl -i -XPOST localhost:8080/api/links \
     -d '{"url":"http://example.com","alias":"abc"}'
curl -i localhost:8080/abc
```

Cheat sheet

```
endpoint
  .post
  .in("api" / path[UUID]("id"))
  .in(query[String]("query"))
  .securityIn(header[String](HeaderNames.Authorization))
  .out(statusCode(StatusCode.Created))
  .out(jsonBody[Entity])
  .out(header[String](HeaderNames.Expires))
  .errorOut(oneOf[NotFound | BadRequest](notFound,badRequest))
  .handleSecurity(apiKey => authService.authorizeUser(apiKey))
  .handle(user => (id, query) =>
     Right((Entity(...), "Mon, 18 Jul 2016"))
  )
```



/ CHECK · QUICK POLL

Which thread runs *service.createLink* for a request?

1 · A Netty event-loop thread

2 · A new virtual thread per request

3 · A fixed pool of 200 threads

4 · The ExecutionContext you pass

sttp: HTTP client

```
val backend =
  DefaultSyncBackend() // direct-style (alternatively: Pekko HTTP, cats-effect, ZIO)
    .pipe(Slf4jLoggingBackend(_)) // logging
    .pipe(OpenTelemetryTracingBackend(_, openTelemetry)) // `traceparent` header in outgoing calls
    .pipe(PrometheusBackend(_)) // metrics

val response: Seq[MyResponse] =
  basicRequest
    .post(uri"$base/example?q=$query")  // safe query param interpolation
    .body(asJson(MyRequest("example")))
    .response(asJsonOrFail[MyResponse]) // handling non-200 responses and JSON decoding
    .send(backend)
    .body
```



/ STEP 2 · STTP + OX

Ox: safe direct-style concurrency, streaming and resilience

Scala 3, JDK 21+, virtual threads: blocking is cheap, so plain calls replace Futures and effect types.

High-level concurrency
`par` · `raceSuccess` · `timeout`

Structured concurrency
`supervised` · `fork` · `useInScope` · `OxApp`

Resilience
`retry` · `repeat` · `Schedule` · `RateLimiter` · `CircuitBreaker`

Streaming
`Flow` · `mapPar` · `throttle` · `merge`

Communication
`Channel` · `select` · `Actor`

Errors and utilities
`either` · `.ok()`

In this lab: **par**, **timeout**, **retry**, **Flow**, **fork**



Ox: concurrency, resilience

```
val (user, posts) = par(fetchUser(), fetchPosts())

val fastest = raceSuccess(fromPrimary(), fromReplica())

val userOption =
  timeoutOption(1.second):
    fetchUser()

val user =
  retry(Schedule.exponentialBackoff(100.millis).maxRetries(4)):
    fetchUser()

repeat(Schedule.fixedInterval(1.second)):
  pollQueue()

val limiter = RateLimiter.fixedWindowWithStartTime(2, 1.second)
limiter.runBlocking:
  fetchUser()
```



/ STEP 2 · 15 MINUTES

Your turn: a client for a flaky dependency

steps/2-resilience.md

1. **PreviewClient.fetchTitle**: GET /title (TitleResponse), retries (backoff from 50 ms, max 3 retries), 1 s in total, otherwise None
2. **PreviewClient.checkSafety**: GET /safety (SafetyResponse), retries, 2 s in total, otherwise Left(Unavailable)
3. **PreviewClient.preview**: both in parallel

FakePreviewServer: the URL decides

URL has	Behaviour
flaky	first 2 calls to each endpoint: 503
slow	/title takes 5 s
lag	every call takes 500 ms
down	/safety always 503
malware	/safety says unsafe

Stretch: a CircuitBreaker around the safety check

Cheat sheet

```
basicRequest
  .get(uri"$baseUri/example?q=$query")
  .response(asJsonOrFail[Example])
  .send(backend)
  .body

retry(Schedule.exponentialBackoff(50.millis).maxRetries(3)) {…}
timeout(1.second) {…}       // throws on timeout
timeoutOption(1.second) {…} // Option[T]
val (a, b) = par(opA, opB)  // both at once
```



/ CHECK · PREDICT THE OUTPUT

How long does this take, and what does it print?

```
par(
  { sleep(500.millis); throw RuntimeException("boom") },
  { sleep(2.seconds); println("done"); 1 }
)
```

1 · 2 seconds, prints "done", then throws

2 · 0.5 second, throws, never prints

3 · 0.5 second, throws, prints in the background

Ox: flows

```
List(1, 2, 3)
  .mapPar(4)(…)

Flow.fromIterable(1 to 8)
  .throttle(4, 1.second)
  .mapPar(4)(…) // like Pekko Streams or fs2, but without Future/IO wrapping
  .filter(…)
  .runToList()  // lazy - evaluated when run
```



/ STEP 3 · 15 MINUTES

Your turn: a bulk import and a background job

steps/3-flow.md

1. **LinkService.importLinks**, behind POST /api/links/import: create the links at most 10 per second (throttle) and at most 4 at a time (mapPar), with a Flow; keep the order of the URLs
2. **TitleRefresher.start**: a fork that, every interval, takes the links without a title (repo.findWithoutTitle), fetches their titles 4 at a time and stores them (repo.updateTitle)

Stretch: what would change with mapParUnordered instead of mapPar?

Cheat sheet

```
supervised: // concurrency scope, provides implicit `Ox`
  fork: // daemon thread: cancelled at the end of scope
    doSomething()

Flow.fromIterable(List(1, 2, 3))
Flow.tick(1.second) // emits every second, forever

flow.throttle(10, 1.second) // at most 10 per second
flow.map(element => ...)
flow.mapPar(4)(element => ...) // 4 at a time, in order
flow.mapConcat(element => List(...)) // 1 → many
flow.runToList() // List[T]
flow.runDrain() // Unit, runs until cancelled
```



/ CHECK · PREDICT THE OUTPUT

What gets printed, in which order?

```
supervised:
  forkUser { sleep(1.second); println("A") }
  fork { sleep(2.seconds); println("B") }
  println("C")
```

1 · C

2 · C, A

3 · C, A, B

Parlance: functional SQL

Raw SQL

```
xa.connect: …  // DbCon: auto-commit

xa.transact:   // DbTx: commit or roll back
  sql"""
    INSERT INTO users (id, name) VALUES ($id, $name)
  """.update.run()

  sql"""
    SELECT id, name, age FROM users WHERE age > $minAge
  """.query[User].run() // Vector[User] - auto-derived

  val fragment = sql"WHERE age >= $minAge"
  sql"""
    SELECT id, name FROM users $fragment
  """.query[(Int, String)].iterator // streaming
```

Entities and CRUD repositories

```
@Table(SqlNameMapper.CamelToSnakeCase)
case class User(@Id id: Int,name: String,email: String)
  derives EntityMeta

case class UserCreator(name: String, email: String)
  extends CreatorOf[User] derives DbCodec

val userRepo = Repo[UserCreator, User, Int]()

xa.transact:
  userRepo.findById(1)  // Option[User]
  userRepo.findAll      // Vector[User]
  val bob = userRepo.create(UserCreator("Bob", "b@x.y"))
  userRepo.deleteById(bob.id)
```



Parlance: functional SQL

Query builder

```
userRepo.query
  .where(u => u.age >= 18 && u.name === "Alice")
  .orderBy(_.age, Desc)
  .limit(10)
  .run()  // Vector[User]

orderRepo.query
  .select(o => (customer = o.customer,
                total = Expr.sum(o.amount)))
  .groupBy(_.customer)
  .having(_.total > 250)
  .run()  // Vector[(customer: String, total: Int)]

pageRepo.query.where(_.id === 1).increment(_.views)
pageRepo.query.where(_.id === 2).update((title = "ABC"))
```

Relationships, ORM-style

```
object Author:
  val books = hasMany[Author, Book](_.id, _.authorId)

object Book:
  val author = belongsTo[Book, Author](_.authorId, _.id)

val author: Author =
  authorRepo.query
    .whereHas(Author.books)(_.title === "Dune")
    .firstOrFail()
val books: Vector[Book] = author.load(Author.books)

authorRepo.update(author.copy(name = "John Smith"))
```



/ STEP 4 · 15 MINUTES

Your turn: implement SqlLinkRepository

steps/4-parlance.md

1. **SqlLinkRepository.find**:
   findById(…) / SELECT … FROM … WHERE …
2. **SqlLinkRepository.insert**:
   insertAllIgnoring(…) / INSERT…ON CONFLICT DO NOTHING
3. **SqlLinkRepository.incrementVisits**:
   query.where(…).increment(…) / UPDATE … SET … WHERE …

```
scala-cli test . --test-only '*Step4*'
```

Stretch: try *both* Query Builder and SQL Interpolator

Cheat sheet

```
xa.connect {…}   // auto-commit
xa.transact {…}  // transaction, rolls back on error

repo.count          // Long
repo.existsById(1) // Boolean
repo.findAll        // Vector[Entity]
repo.findById(1)    // Option[Entity]
repo.create(Entity(…))
repo.insertAllIgnoring(List(Entity(…))) // Int
repo.save(Entity(…))
repo.query.where(…).run() // Vector[Entity]
repo.query.where(_.a === a).increment(_.b, 5) // Int
repo.query.where(_.a === a).update((b = 5))   // Int

sql"SELECT a FROM entity".query[Entity].run() // Vector[Entity]
sql"UPDATE entity SET a = $a".update.run() // Int
sql"INSERT INTO entity (a) VALUES ($a)".update.run() // Int
```



/ CHECK · QUICK POLL

xa.transact blocks the thread for the whole transaction. Is that a problem?

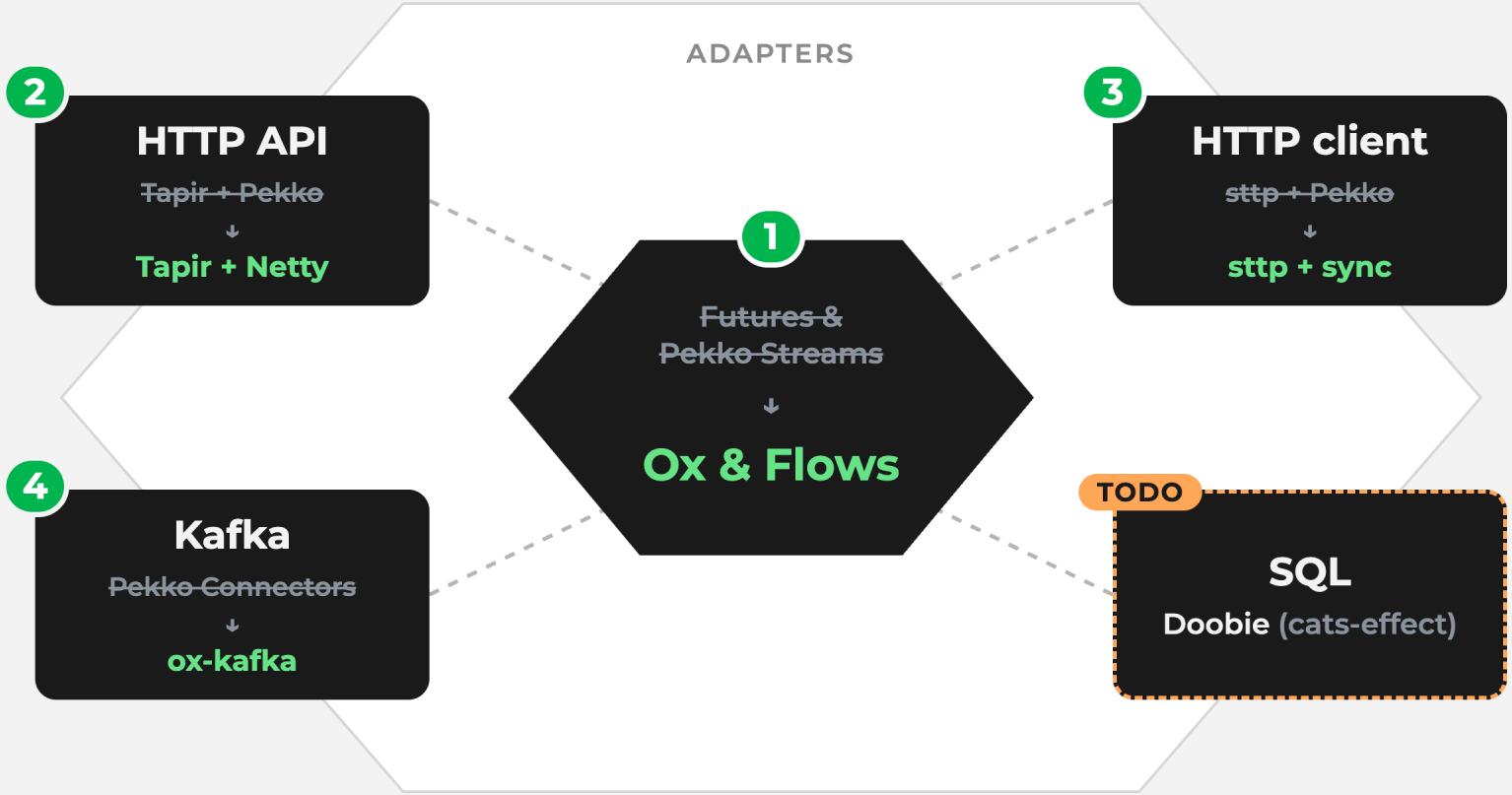
1 · Yes: wrap it in a Future

2 · Yes: use a separate JDBC thread pool

3 · No: it's a virtual thread

4 · No: H2 is in-memory

Case study: migration Pekko → direct-style



SCOPE

~60k lines of code

4 microservices

RESULTS

−8% memory usage

−2% API latency

−2% lines of code

Your turn: rewrite AnalyticsService in direct style

steps/5-migration.md

AnalyticsService.report is Future-based: flatMap, recover, a hand-made timeout (Promise + scheduler) and an ExecutionContext.

1. **AnalyticsService.report**: Future[Either[…]] becomes Either[NotFound, LinkReport], with the same behaviour: both clients in parallel, referrers slower than 1 s skipped (Nil)
2. **ReportApi**: the compiler points at it; drop the .get()
3. **WeeklyDigest**: its callers expect a Future, so keep its API, and run the direct-style calls on virtual threads

One layer at a time: the clients stay Future-based (a shared library you can't change yet). Wait for them with .get().

Cheat sheet

```
future.get() // blocks (cheap on a virtual thread)

val virtualThreadEC = ExecutionContext.fromExecutorService(
  Executors.newVirtualThreadPerTaskExecutor()
)
Future(…)(using virtualThreadEC)
```



/ THREE THINGS TO TAKE HOME

1 **Plain code on virtual threads, with structure**

Blocking is cheap. Ox scopes make sure no fork outlives its owner.

2 **Concurrency, resilience - a function call**

Ox's par, mapPar, timeout, retry

3 **Migrate one layer at a time**

.get() on the Futures below, Future(...) on virtual threads for the callers above.



Thank You

Piotr Janczyk

pjanczyk@virtuslab.com

VirtusLab Scala Stack

vss.virtuslab.com